TP : Génération et distribution de certificats

Rushed Kanawati

Résumé

L'objectif de ce TP est de se familiariser avec les fonctions de productions et de distribution de certificats en utilisant la bibliothèque `openssl`. Nous illustrons le processus à travers la certification d'un service web. Les thèmes abordés sont :

— Génération de certificat auto-signé

— Installation d'une autorité de certification simple

Introduction

`SSL` (Secure Socket Layer) est un ensemble de protocoles développé par la société Netscape Communication Corporation pour permettre de la communication sécurisée en mode client/serveur pour des application réseaux utilisant TCP/IP. Le protocole TLS (Transport Layer Security) est une évolution de SSL réalisé par l'IETF et qui sert de base à HTTPS par exemple. Pour communiquer avec un serveur web en https, ln client le client récupère la clé publique du service puis il s ?en sert pour chiffrer une communication avec le service en SSL Par cette communication, ils échangent une clé symétrique pour chiffrer leurs échanges futurs. En résumé, la clé publique sert juste a chiffrer un échange pour obtenir une **clé de session** beaucoup plus efficace en terme de performances.

`OpenSSL` est une implantation libre des protocoles SSL qui offre :

— une bibliothèque de fonctions pour réaliser des applications client/serveur sécurisées s'appuyant sur SSL,

— un ensemble de commande en ligne qui permettent entre autres de :

  — forger des clés RSA, DSA

  — créer de certificats

  — calcul erd'empreinte (MD5, SHA, ...)

OpenSSL : Opérations de base

— `$ openssl genrsa -out <fichier_rsa.priv> <size>` : génère la clé taille size.

— `$ openssl rsa -in <fichier_rsa.priv> -text -noout` pour visualiser la clé.

— `$ openssl rsa -in <fichier_rsa.priv> -des3 -out <fichier.pem>` : chiffre la clef privé RSA avec l'algorithme DES3. Vous pouvez utiliser DES, 3DES, IDEA, etc.

— `$ openssl rsa -in <fichier_rsa.priv> -pubout -out <fichier_rsa.pub>` : stocke la partie publique dans un fichier à part (création de de la clé publique associée à la clef privée dans le fichier fichier.pem).

— `$ openssl rsautl -encrypt -in <fichier-clair> -inkey <cle> -out <fichier-out>` : pour chiffrer un fichier avec une clé RSA.

— `$ openssl rsautl -decrypt -in <fichier> -inkey <cle> -out <fichier-out>` : pour déchiffrer un fichier avec une clé RSA.

— `$ openssl req -x509 -newkey rsa:<size> -keyout <fichier-key> -out <fichier-cert> -days n -nodes` :
`req` invoque le module émission de certificat. L'argument `-x509` accolé à `req` génère un certificat auto-signé. L'argument `-newkey` génère une clé privée, ici avec l'algorithme asymétrique RSA. `-keyout` indique le fichier dans lequel stocker la clé privée. `-out` indique le fichier dans lequel stocker le certificat. `-days` indique la durée de validité du certificat. Enfin, l'argument `-nodes` indique que la clé privée n'est pas chiffrée.

— `$ openssl enc <-algo> -in <claire.txt> -out <chiffre.enc>` : pour le chiffrement de `claire.txt` avec l'algorithme spécifié (`openssl enc --help` pour avoir la liste des possibilités

— `$ openssl rsautl -sign -inkey <ras.priv> -in <fichier.txt> -out <fic.sig>` : pour générer une signature.

— `$ openssl rsautl -verify -pubin -inkey <rsa.pub> -in <fic.sig> : pour vérifier une signature`.

Jouer avec RSA

1. Générer une paire de clés RSA de taille 512 dans un fichier `my-key.pem`. Exporter la clé public dans un fichier `my-pubkey.pem`
2. Visualiser la clé stocké dans `my-key.pem`
3. Générer un fichier `toto` qui contient votre nom et chiffrer le en utilisant la clé publique. Utiliser l'option `-pubin` pour indiquer que vous utilisez une clé publique.
4. Déchiffrer le fichier chiffré avec la clé privée.
5. Signer le fichier `toto` avec votre clé privée et vérifier la signature avec la clé publique

Certification d'un service web

Nous allons ici générer un certificat auto-signé pour un service web à installer sur votre machine.

1. Installer le package `apache2` si ce n'est pas installé.
2. Créer un répertoire `ssl` dans `/etc/apache2`. Générer dans ce répertoire un certificat auto signé avec l'algorithme `RSA` avec un clé de taille 4096 bits. le certificat devrait avoir une validité d'un mois. Dans cette seconde étape, il faut renseigner les attributs du certificat de façon interactive. L'attribut auquel il faut être particulièrement attentif est le Common Name qui doit être fixé à la valeur du FQDN du service à certifier. Ici il s'agit d'un serveur web donc nous fixons l'attribut avec la valeur www.lab.local. L'étape 3 est invisible mais il s'agit bien de la signature automatique de la CSR générée précédemment pour obtenir le fichier `cert.pem`.

3. Configuration du service apache2 : Dans le fichier `/etc/apache2/sites-available/default-ssl.conf` fixer les bonnes valeurs des deux variables `SSLCertificateFile` et `SSLCertificateKeyFile`. Puis on active le module SSL d'Apache en exécutant la commande `a2enmod ssl`. Puis on active l'écoute sur le port 443 par défaut en exécutant la commande `a2ensite default-ssl`.
4. Activer le service `apache2` puis se connecter au site web. Justifier le message d'alerte affiché.
5. Accepter le certificat. vérifier que ce certificat est bien enregistré dans le liste des certificats valides pour votre navigeteur (section préférence / vie privé)

Installation d'une autorité de certification

1. Créer les répertoires suivants : `/opt/ca /opt/ca/csr /opt/ca/certs` dans lesquels nous allons installer notre CA.
2. Dans `/opt/ca` générer une clé privée pour le CA. : `openssl genrsa -out CA-key.pem 4096`
3. Créons ensuite le certificat auto-signé qui amorce la pompe de l'autorité de certification. Ce certificat est amené à être distribué dans les magasins des clients qui vont se connecter au site. Voici la commande :
   `openssl req -new -key CA-key.pem -x509 -days 1000 -out CA-cert.pem`. Justifier les arguments de cette commande.
4. Créer une nouvelle clé privée pour le service web et stocker la dans `www-key.pem` dans `/etc/apache2/ssl`
5. Créer une requête de signature de certificat (CSR) à être signée par la CA :
   `openssl req -new -key www-key.pem -out www-cert.csr`
6. Copier cette requête dans : `/opt/ca/certs`. Aller dans ce répertoir et signer la CSR :
   `openssl x509 -req -days 365 -in ../csr/www-cert.csr -CA ../CA-cert.pem -CAkey ../CA-key.pem -CAcreateserial -out www-cert.pem`
7. Recopier le certificat signé et le certificat de la CA dans `/etc/apache2/ssl`.
8. Ajouter dans le fichier de configuration du serveur web la ligne `SSLCertificateChainFile /etc/apache2/ssl/CA-cert.pem`
9. Pour éviter les alerts, importer le certificat du CA dans le magasin de certificat de votre navigateur.
10. Bonus : avec un binôme, essayer d'installer deux CA en situation d'authentification mutuelle.